Using AI to Analyze Jenkins Build Failures

A Step-by-Step Guide Using OpenAI and Anthropic APIs

Jenkins, OpenAI, and Anthropic Claude are trademarks of their respective owners.
This guide is not affiliated with or endorsed by these organizations.

---

1. Before You Begin

This guide uses what you might call a “bull in a China shop” approach.

It is also intentionally pedantic.

We are not trying to be clever or minimal.
We are not optimizing for elegance, reuse, or production readiness.

We are optimizing for clarity and proof: getting from a failed Jenkins build to a useful AI-generated explanation as quickly and transparently as possible.

This guide shows one way to send Jenkins build logs to an AI service and return a short failure analysis directly to the Jenkins console.

We are trying to make each step visible, understandable, and repeatable.

Once it works, then we can make it better.

This is not a full Jenkins installation guide. Jenkins environments vary widely. The examples in this guide use a local Docker-based Jenkins setup for demonstration purposes.

---

2. What You Will Build

The Problem

When a Jenkins build fails, the information you need is usually already there. It's just buried.

Let's dig that out.

By the end of this guide, you will have:

- a Jenkins job that fails in a controlled way
- a pipeline that captures the failure log
- a connection to OpenAI or Anthropic
- a short AI-generated explanation of the error (and potential fixes) returned to the Jenkins console

---

3. Configure Your System

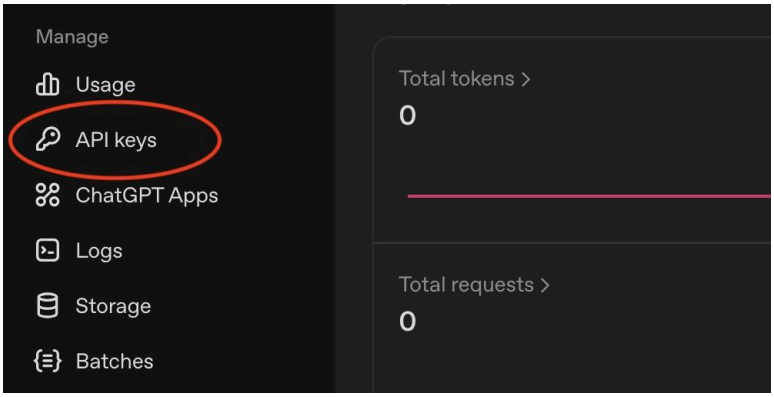
3.1 Create AI provider accounts

For this exercise, you will need to create and fund platform accounts because we'll be making API calls instead of posting inputs and prompts to the dashboards. Funding can be minimal. What you'll be doing here does not consume many tokens.

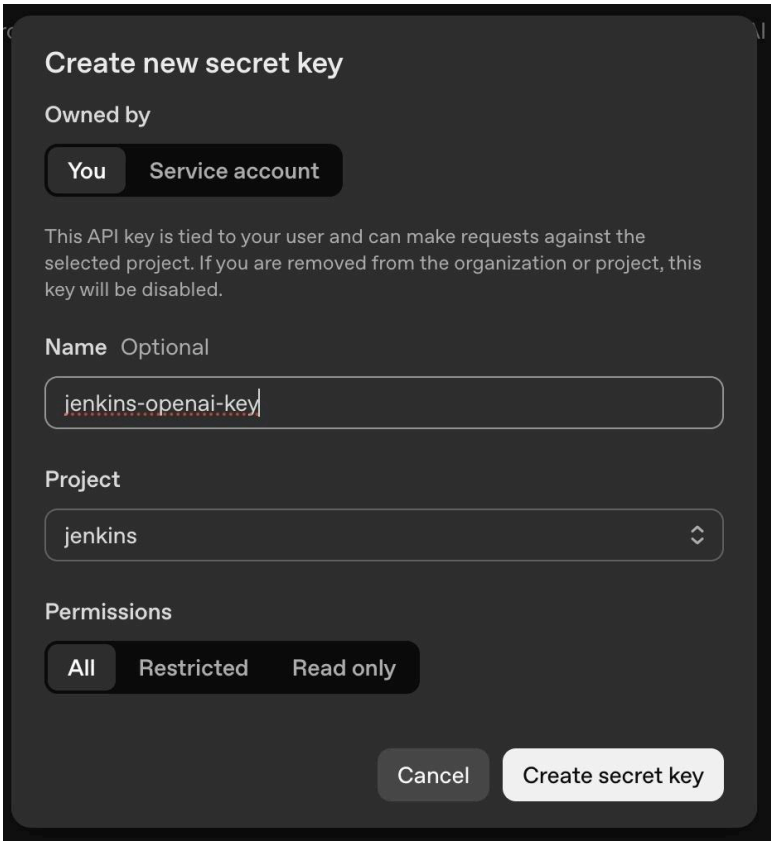
1. Create an OpenAI platform account at **https://platform.openai.com/home**
2. Create an Anthropic platform account at **https://platform.claude.com/dashboard**
3. Add billing or credits to each account.

3.2 Create API keys

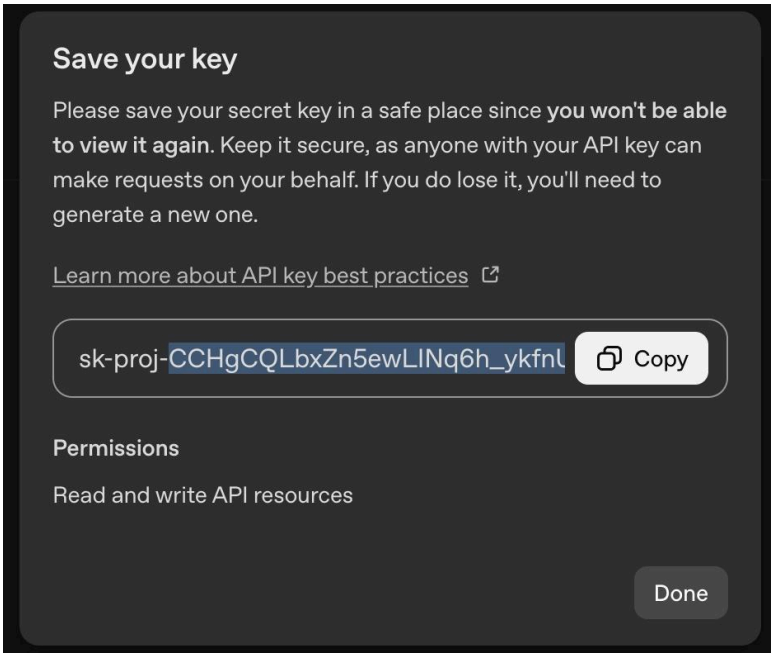
1. Create an API key in OpenAI
    a. Look for the **API Keys** link in the left hand navigation pane of the dashboard;



b. Click the **Create new secret key** button to open this dialog box



c. Give the key a name and add it to a project (optional). Click the **Create secret key** button to reveal the key code



d. Click **Copy** and save the key in a safe place. You will be entering the key into the Jenkins credentials manager later in the exercise

2. Create an API key in Anthropic
    a. The steps for Claude are quite similar. Follow the instructions in the dashboard.

3.3 Prepare your local environment

1. Make sure Docker is installed
2. Make sure Docker Compose is available
3. Make sure you can access Jenkins

This guide does not walk through every possible Jenkins installation method. A sample local setup is included in the repository as a bash script along with `docker-compose.yaml`.

3.4 Clone the repository

1. Clone the repository from; `https://github.com/jayblanchard/jenkins-ai-log-analysis.git`
2. Review the included files
3. Confirm you have the sample pipeline files and setup files

---

4. Start Jenkins

(Only if you do not have a Jenkins instance running)

4.1 Start the container

1. Run the provided startup command or `docker compose up -d`
2. Confirm the Jenkins container is running

4.2 Unlock Jenkins

1. Open Jenkins in your browser
2. Retrieve the initial admin password
3. Complete the first login steps and **CHANGE THE PASSWORD**

4.3 Install required plugins

1. Install the suggested plugins or required plugins
2. Confirm Pipeline is available
3. Confirm Credentials Binding is available

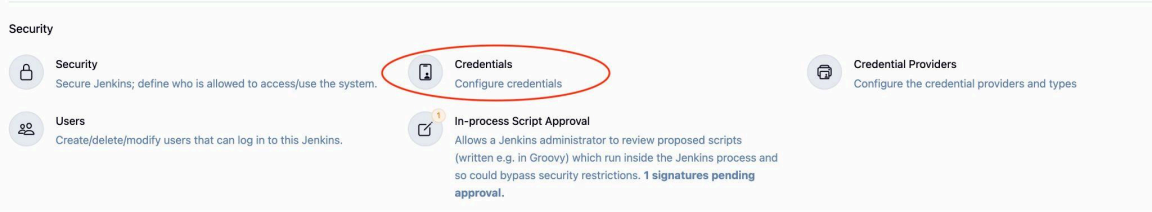
---

5. Configure Jenkins Credentials

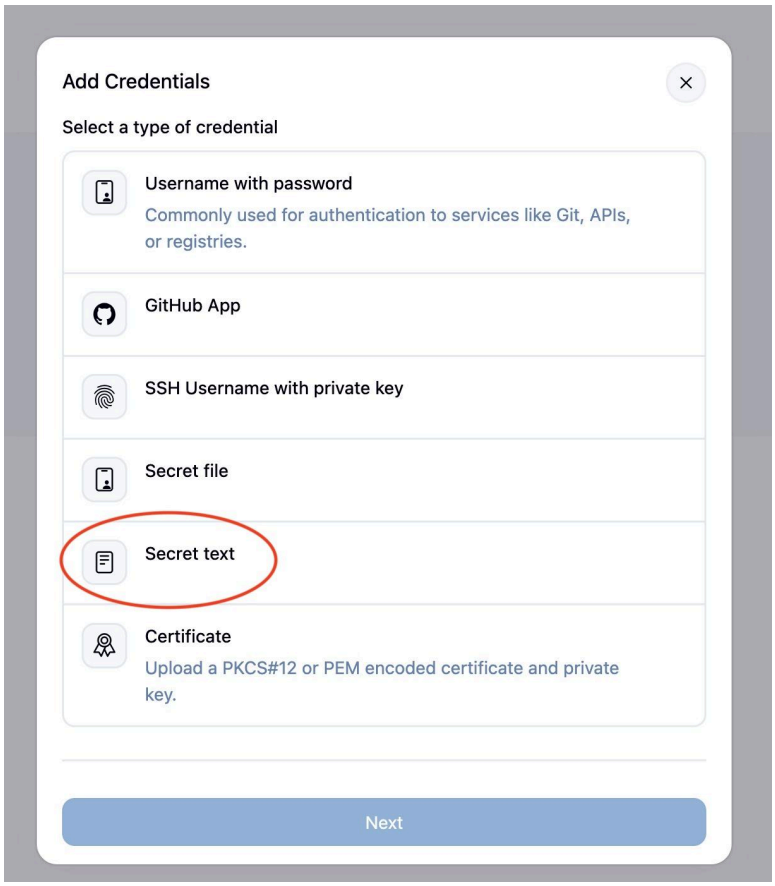
5.1 Add the OpenAI key

1. Open Jenkins credentials
   Click the **Settings** icon in the top right hand corner

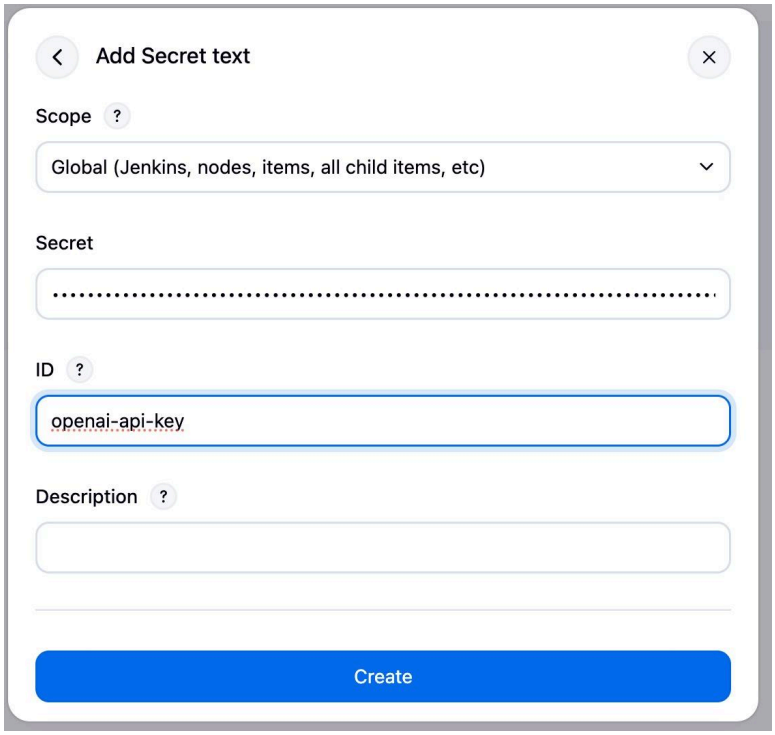
   Select **Credentials** in the *Security* section of the Jenkins management interface



2. Create a new Secret text credential
   Click the **+ Add Credentials** button
3. Enter the OpenAI API key
   Select **Secret text**



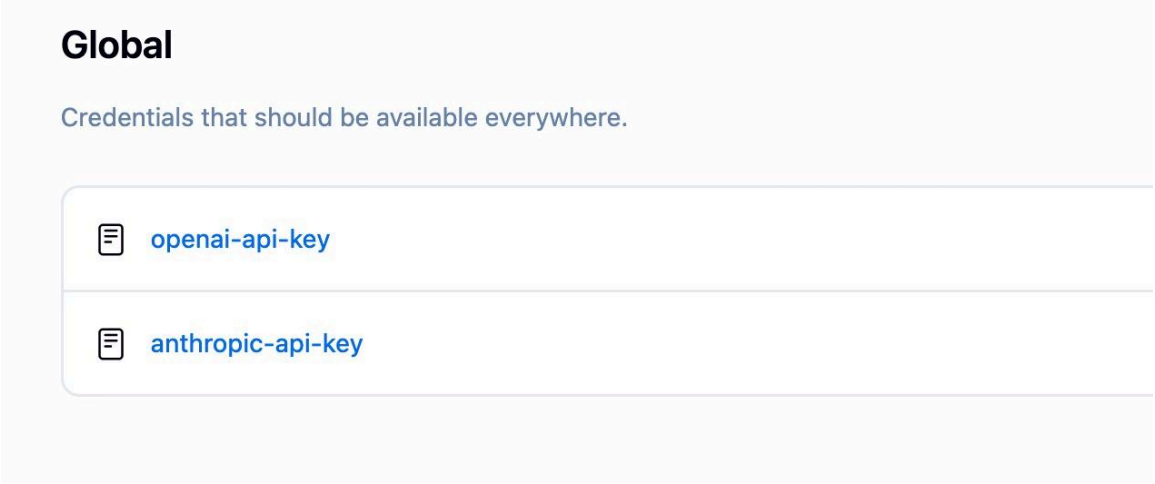
Enter the **Secret** that you got from OpenAI



4. Set the credential ID to `openai-api-key` and click **Create**

5.2 Add the Anthropic key

1. Follow the same steps and set the credential ID to `anthropic-api-key`



---

6. Configure the Jenkins Environment

If you built the Jenkins Docker container using the included **Dockerfile** and **docker-compose.yaml**, you will already have Python3 installed in your Jenkins instance.

If not, you will need to ensure that Python is available inside the Jenkins container. This guide uses Python to parse the responses returned from the AI APIs.

6.1 Add Python to the Jenkins image

1. Review the provided Dockerfile
2. Build the custom Jenkins image
3. Restart Jenkins with the updated image
4. Confirm `python3` is available inside the container

This requirement surfaced during initial testing, where the Jenkins container did not include Python by default.

The smallest things will get you.

---

7. Create the First Test Job

The Jenkins console log in this example is intentionally contrived and designed to fail. We want to make sure we give Anthropic/OpenAI a chance to return identification of the failure and propose potential solutions for the fixes.

7.1 Create the Anthropic job

1. Create a new Pipeline job
2. Name it `initial-pipeline-anthropic`
3. Paste in the provided Jenkinsfile Jenkinsfile-Anthropic
4. Save the job

7.2 Run the job

1. Run the pipeline
2. Open the console output
3. Confirm the job fails
4. Confirm the AI analysis appears in the Jenkins console

7.3 Create the OpenAI job

1. Create a second Pipeline job
2. Name it `initial-pipeline-openai`

3. Paste in the provided Jenkinsfile Jenkinsfile-OpenAI
4. Save the job

7.4 Run the job

1. Run the pipeline
2. Open the console output
3. Confirm the job fails
4. Confirm the AI analysis appears in the Jenkins console

Did you run into any errors?
Did you have API key issues?
Or did everything return as expected?

Regardless of the outcome — **CONGRATULATIONS**!

You just completed your first Jenkins interaction with two different AI APIs.

Take a few minutes to review the code and make sure you understand the flow.

It isn't complex. As mentioned earlier, we went full "bull in a China shop" to get something working. This is about clarity and proof, not production readiness.

---

8. Understand What the Pipeline Is Doing

8.1 The build stage

- creates a controlled failure
- writes output to `build.log`

8.2 The analysis stage

- reads `build.log`
- sends the log to the AI provider
- prints the response to the Jenkins console

8.3 The final result

- the build still fails
- the AI analysis does not hide the real build result
- Jenkins still behaves normally

9. Compare the Results

There are differences in the returned information. Please take note of the information presented and more importantly, how it is presented. Is the voice “engineering” or is it more casual? Does either response include assumptions or guesses that are not directly supported by the log?

Understand that neither response is better than the other. The goal is to understand how each behaves so you can decide what fits your workflow.

9.1 Anthropic output

```
+ echo ===== AI LOG ANALYSIS (CLAUDE) =====

===== AI LOG ANALYSIS (CLAUDE) =====

+ python3 -

Summary:

The build failed due to an unresolvable dependency (`foo-bar:1.2.3`)
during the dependency resolution phase.

Likely Cause:

The dependency `foo-bar:1.2.3` could not be found — it may not exist
in the configured repositories, the version may be incorrect, or the
repository may be unreachable.

Relevant Log Lines:

- `ERROR: Failed to resolve dependency foo-bar:1.2.3`

Suggested Fixes:

1. Verify that `foo-bar:1.2.3` exists in your configured package
repositories (e.g., check Maven Central, npm registry, internal
Artifactory, etc.).

2. Check if the version `1.2.3` is correct — it may have been
unpublished, yanked, or the version string may be a typo.

3. Ensure the Jenkins build agent has network access to the relevant
repository (check proxy settings, firewall rules, and DNS
resolution).
```

```
4. **Possible additional check:** Review whether `foo-bar` has been
renamed or replaced by a different package, and update the dependency
declaration accordingly.

5. **Possible additional check:** Investigate the `WARNING:
deprecated package detected` message — a deprecated transitive
dependency may have been removed from the registry, causing the
resolution failure.

+ echo ===== END AI LOG ANALYSIS (CLAUDE) =====

===== END AI LOG ANALYSIS (CLAUDE) =====
```

9.2 OpenAI output

```
+ echo ===== AI LOG ANALYSIS (OPENAI) =====

===== AI LOG ANALYSIS (OPENAI) =====

+ python3 -

Summary:

Dependency resolution failed during the build process due to a
missing or problematic package.

Likely Cause:

Missing or unavailable dependency foo-bar:1.2.3.

Relevant Log Lines:

- ERROR: Failed to resolve dependency foo-bar:1.2.3

- WARNING: deprecated package detected

Suggested Fixes:

1. Verify that foo-bar:1.2.3 exists in your dependency repository or
registry.

2. Check your build configuration for typos or incorrect version
specification for foo-bar.

3. Replace foo-bar:1.2.3 with a supported or non-deprecated version
if available.
```

```
4. Possible additional check: Ensure your network connection to the
dependency repository is working.

+ echo ===== END AI LOG ANALYSIS (OPENAI) =====

===== END AI LOG ANALYSIS (OPENAI) =====
```

9.3 What we learned

- both services worked
- both found the main failure
- Anthropic tended to provide broader guidance
- OpenAI tended to stay a little tighter to the log evidence

---

10. Common Issues I Hit

While creating this guide, I started with a fresh installation of all of the tools and I ran into some issues. In the interest of fairness, I wanted to share those issues with you.

- Docker context issues with Colima - I installed Colima for my Docker environment and completely whiffed when I tried to build and start a container.
- Jenkins login reset problems - I clicked through too quickly and missed grabbing the default admin password for Jenkins. I also missed installing the default plugins because I was in a rush.
- Bash-only syntax causing `Bad substitution -` we've all been here before.
- Jenkins script security blocking JSON parsing. For this particular installation of Jenkins I had to specifically allow the Jenkinsfile to perform the JSON parsing operations.
- Missing `python3` in the Jenkins container - as I said before, this one surfaced mid-run, when Jenkins helpfully informed me that `python3` did not exist.

None of these were particularly hard to fix. They were just the kind of small things that slow you down if you don't expect them.

The smallest things will get you.

---

11. Next Steps

At this point, you have a working example. The next step is to make it more useful for your own environment.

Here are a few directions you can take:

- **Test with a messier Docker build failure.**
  Right now, the example is clean and predictable. Try introducing a more realistic failure using a Docker build:
  - invalid base image
  - missing file in a COPY command
  - package installation failure
  - authentication failure when pulling an image

This will help you understand how the AI handles more complex and noisy logs. You can also test with logs that have Maven build failures, connection refusals to external resources, etc.

- **Move prompts into separate files.**
  The prompt is currently embedded directly in the pipeline.
  - You can make this easier to maintain by moving it into a separate file and loading it at runtime.

This also allows you to experiment with different prompts without modifying your pipeline code.

- **Move provider logic into helper methods**
  The OpenAI and Anthropic calls are written inline in the pipeline for clarity.
  - You can extract this logic into reusable methods to reduce duplication and make your pipelines easier to read and maintain.

- **Use a Jenkins shared library**
  - If you plan to use this across multiple pipelines, consider moving the logic into a Jenkins shared library.

This allows you to reduce your pipeline to a simple call, for example:

```
analyzeBuildWithAI()
```

- **Send results to Slack or email**
  Instead of only printing the results to the Jenkins console, you can send the analysis to:
  - Slack
  - Email
  - other notification systems

This can help your team see and act on failures more quickly.

- **Trim logs before sending**
  In this example, the full log is sent to the AI service. In a real environment, you may want to:
  - limit the number of lines sent
  - filter out known noise
  - send only the relevant portion of the log

This can reduce cost and improve response quality.

- **Support more realistic build scenarios**
  Expand beyond the initial example:
  - test failures
  - integration failures
  - deployment errors
  - multi-stage pipelines

This will help you understand how the approach performs in different situations.

---

12. Final Notes

Please keep the following in mind:

- Every Jenkins environment is different.
- Different AI models may respond differently, even when given the same input.
- Use this guide as a starting point, not a universal implementation plan.
- Test all changes in a non-production environment before applying them more broadly.

Start simple. Get it working. Then make it fit your world.